Autogen Single Agent Formatted

```
import autogen
from typing import Dict, List

# ------------------------------
# Config
# ------------------------------
MODEL_FOR_REASONING = "gpt-4o"         # good balance of quality/speed
MODEL_FOR_LIGHT = "gpt-4o-mini"        # cheap/faster for smaller steps
from dotenv import load_dotenv
import os
import openai

# Load API key from file and set as environment variable
def set_api_key(file_path='api_key.txt'):
    with open(file_path, 'r') as file:
        api_key = file.read().strip()
    os.environ["OPENAI_API_KEY"] = api_key

# Set the API key
set_api_key()

# Configure OpenAI with the environment variable
openai.api_key = os.getenv("OPENAI_API_KEY")

from dotenv import load_dotenv
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY")
# Configuration for the LLM
config_list = [
    {
        "model": "gpt-4",
        "api_key": api_key
    }
]

llm_config = {
    "config_list": config_list,
    "temperature": 0.7,
    "timeout": 120,
}


# Create a single assistant agent
assistant = autogen.AssistantAgent(
    name="CodeAssistant",
    system_message="""You are an expert Python programmer. Your capabilities
include:
    1. Writing clean, efficient Python code
    2. Debugging and fixing code issues
    3. Explaining code concepts clearly
    4. Providing best practices and optimization suggestions

    When writing code:
    - Include helpful comments
```

```
    - Follow PEP 8 style guidelines
    - Handle edge cases
    - Write testable code

    Always provide explanations with your code.""",
    llm_config=llm_config,
)

# Create a user proxy agent (represents the human user)
user_proxy = autogen.UserProxyAgent(
    name="User",
    human_input_mode="NEVER",  # Change to "ALWAYS" for interactive mode
    max_consecutive_auto_reply=10,
    is_termination_msg=lambda x: x.get("content",
"").rstrip().endswith("TERMINATE"),
    code_execution_config={
        "work_dir": "coding_workspace",
        "use_docker": False,  # Set to True for safer execution in Docker
    },
    llm_config=llm_config,
)

# Example 1: Simple code generation
def example_code_generation():
    print("\n" + "="*60)
    print("Example 1: Code Generation")
    print("="*60 + "\n")
```

```
    task = """Write a Python function that:
    1. Takes a list of numbers
    2. Removes duplicates
    3. Sorts the list
    4. Returns only even numbers

    Include docstring and example usage."""

    user_proxy.initiate_chat(
        assistant,
        message=task
    )

# Example 2: Code debugging
def example_debugging():
    print("\n" + "="*60)
    print("Example 2: Code Debugging")
    print("="*60 + "\n")

    buggy_code = """
    def calculate_average(numbers):
        total = 0
        for num in numbers:
            total += num
        return total / len(numbers)

    # This code has a bug when passed an empty list
    result = calculate_average([])
```

```
    print(result)
    """

    task = f"""Debug this code and fix any issues:

    {buggy_code}

    Explain what was wrong and how you fixed it."""

    user_proxy.initiate_chat(
        assistant,
        message=task
    )

# Example 3: Code optimization
def example_optimization():
    print("\n" + "="*60)
    print("Example 3: Code Optimization")
    print("="*60 + "\n")

    inefficient_code = """
    def find_duplicates(lst):
        duplicates = []
        for i in range(len(lst)):
            for j in range(i + 1, len(lst)):
                if lst[i] == lst[j] and lst[i] not in duplicates:
                    duplicates.append(lst[i])
        return duplicates
```

```
    """

    task = f"""Optimize this code for better performance:

    {inefficient_code}

    Explain the optimization and show time complexity improvement."""

    user_proxy.initiate_chat(
        assistant,
        message=task
    )

# Example 4: Data processing task
def example_data_processing():
    print("\n" + "="*60)
    print("Example 4: Data Processing Task")
    print("="*60 + "\n")

    task = """Create a Python script that:
    1. Generates 100 random data points (simulating sales data)
    2. Calculates mean, median, and standard deviation
    3. Creates a simple visualization using ASCII art
    4. Identifies outliers (values beyond 2 standard deviations)

    Make it self-contained and runnable."""

    user_proxy.initiate_chat(
```

```
        assistant,
        message=task
    )

# Run examples
if __name__ == "__main__":
    print("AutoGen Single Agent Examples - Code Assistant")
    print("=" * 60)

    # Uncomment the examples you want to run:

    example_code_generation()
    # example_debugging()
    # example_optimization()
    # example_data_processing()

    print("\n" + "="*60)
    print("All examples completed!")
    print("="*60)

# Advanced: Custom function calling
def create_agent_with_functions():
    """Example showing how to give agents custom functions to call"""

    def analyze_sentiment(text: str) -> str:
        """Simulated sentiment analysis"""
        # In production, use actual ML model
        positive_words = ["good", "great", "excellent", "happy"]
```

```
    negative_words = ["bad", "terrible", "awful", "sad"]

    text_lower = text.lower()
    pos_count = sum(word in text_lower for word in positive_words)
    neg_count = sum(word in text_lower for word in negative_words)

    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

def fetch_data(source: str) -> str:
    """Simulated data fetching"""
    return f"Data from {source}: [Sample data points...]"

# Register functions
autogen.register_function(
    analyze_sentiment,
    caller=assistant,
    executor=user_proxy,
    description="Analyzes the sentiment of given text"
)

autogen.register_function(
    fetch_data,
    caller=assistant,
```

```
    executor=user_proxy,
    description="Fetches data from a specified source"
)

# Now the agent can call these functions
task = """Use the available functions to:
1. Fetch data from 'customer_reviews'
2. Analyze the sentiment
3. Provide a summary
"""

user_proxy.initiate_chat(assistant, message=task)
```